

IT9204: Emerging Technologies in AI

Individual Project Assessment Description

Semester:	Semester B, 2025-2026
Instructor:	Husain Naser (Course Coordinator)
Learning Outcomes Covered:	• CILO 1: Critically analyze emerging AI theories, paradigms, technologies, and their potential impact and limitations within evolving contexts. • CILO 2: Evaluate advanced AI techniques to address complex challenges in diverse application domains, considering their ethical, societal, and economic implications.
Weighting:	• Project (Individual) – 50% (not must pass)
Deadlines:	• Project (Individual) – 30th May 2026, 11:55 PM
Notes:	<u>Extensions:</u> • Requests for extensions should be made 48 hours prior to the deadline and sent to the Course Coordinator. Extensions will only be approved for valid reasons with verifiable documentation. <u>Late submissions:</u> • All late submissions (less than 3 days) will be capped at 60%. All late submissions (more than 3 days) will receive 0 grade.



Table of Contents

Project Overview	3
Submission Instructions	4
Suggested Problem Domains.....	5
Project Timeline	7
Topic Approval Form.....	7
Mark Allocation.....	8
Detailed Requirements.....	9
Part A: Data & Knowledge Layer (15 Marks)	9
Part B: Machine Learning Component (15 Marks)	9
Part C: Agent System (20 Marks)	9
Part D: Decision & Reasoning Layer (15 Marks).....	9
Part E: System Integration (10 Marks)	10
Part F: Evaluation & Analysis (15 Marks).....	10
Part G: Demo Video (5 Marks).....	10
Part H: Report Quality (5 Marks).....	10
Part I: Extra Challenging Work (5 Marks).....	11
Appendix I - Suggested Project Scenarios	12
1. Student Success and Academic Advising Assistant	12
2. Employee Attrition and HR Advisory Assistant.....	12
3. Customer Support Ticket Triage and Resolution Assistant.....	13
4. Clinical Risk and Care Guidance Assistant	13
5. Financial Risk and Loan Advisory Assistant.....	14
6. Retail Demand and Inventory Decision Assistant	14
7. Energy Usage and Sustainability Advisory Assistant	15
8. Insurance Claims Screening Assistant.....	15
7. Propose your own	16

Project Overview

This project assesses your ability to design, implement, and evaluate a **complete AI system in the Cloud** that goes beyond isolated models or simple chatbots. You will build an intelligent solution that integrates multiple AI components into a single, coherent solution capable of supporting real-world decision-making.

The system you develop must simulate a realistic scenario where a user interacts with an AI agent + RAG(Grounding) + Cloud ML models to solve a problem that requires information retrieval, data analysis, and predictive reasoning. Your assistant should not rely on a single technique, but instead combine multiple capabilities to produce accurate, grounded, and context-aware responses.

Specifically, your system is expected to:

- Interpret and understand user queries in natural language
- Retrieve relevant information from a document-based knowledge source
- Access and utilize structured data or external tools when required
- Invoke a machine learning model to generate predictions or insights
- Combine all outputs into a clear, justified, and user-friendly response

To achieve this, your solution must include and integrate the following components:

- LLM Layer: A large language model (e.g., Azure OpenAI) used for reasoning, orchestration, and response generation
- RAG Component: A retrieval pipeline that allows the system to search and use external documents as grounding knowledge
- Agent Component: A mechanism that enables the system to decide when and how to use tools, APIs, or models (e.g., tool calling or MCP)
- Machine Learning Component: A predictive model (e.g., built using Azure Machine Learning tools, AutoML, or any other method) deployed and used within the system workflow
- External Tools or Data Access: At least one structured data source or API that the system can query dynamically
- Decision Logic Layer (Critical): A clearly defined strategy that determines when the system should rely on retrieval, tools, or machine learning, rather than treating the LLM as a black box
- Proper analysis of Model Performance, Monitoring, Responsible AI concepts, and Ethical use considerations

The final system should demonstrate end-to-end integration, where all components work together seamlessly. The focus of this project is not only on building individual parts, but on designing a system where each component plays a clear role in solving the problem and improving the quality of the final response.

The emphasis of this project is on system design, integration, and reasoning rather than achieving perfect model performance. Azure gives you intelligent behavior. You are required to design intelligent systems. That's one level higher.

Submission Instructions

This is an individual project. All work must be your own.

- Submit a single ZIP file named IT9204_StudentID (e.g., IT9204_20241234) containing:
 - Code file(s): Any files with code, outputs, and inline explanations.
 - Project report: .docx format, with a cover page including your full name and student ID.
 - Demo video link: .mp4 format, 3-5 minutes, screen recording with audio narration (might be replaced with live demo. Instructions on the demo part will be put on Moodle. In case of video submission, you may submit a link to a video on OneDrive or any other streaming platform marked as Unlisted, and ensure it is accessible from the link).
 - Dataset: A download link to any data set you may have used (online data set, or one that you have created)
- The solution is expected to utilize Azure AI Foundry and Azure services where appropriate. Supporting tools such as VS Code, notebooks, or SDK-based development may be used.
- Upload the ZIP file to the Moodle submission link provided for the project

Suggested Problem Domains

Students must choose one complete project scenario, or propose a custom project of equivalent scope for approval. Each approved project must clearly contain:

- A document knowledge layer for RAG
- A structured data or API tool layer
- A predictive ML layer
- An agent layer that decides how to use these components
- A final integrated response that combines evidence, prediction, and explanation

A project that only “chats with documents” or only “queries a database” is not sufficient.

Choosing a good domain where the pipeline suggested would be logical and useful is the first challenge, and a proper analysis will give you a higher chance of success when building your full solution. You need enough analysis and prior planning on:

- What documents you will use for RAG?
- What structured data or API will you query as tools?
- What Machine Learning / AutoML target you can train?
- What the agent is expected to do end-to-end?

Thus, before choosing a problem domain, please make sure to do your own research on its feasibility, data availability, and use case.

You may select from the following suggested domains or propose your own topic. Should you have any Custom topics in mind, it must be approved by the instructor during the Topic Approval period (see below).

Scenario	Problem Focus	RAG (Documents)	Data / Tools	ML Task	Expected System Behavior
Student Success Assistant	Identify at-risk students and provide academic guidance	Policies, regulations, module guides	Student records, attendance, grades	Predict risk of failure	Combine policy + data + prediction to recommend actions
HR Attrition Assistant	Analyze employee turnover risk	HR policies, performance guidelines	Employee records, performance data	Predict attrition risk	Explain risk and suggest HR actions based on policy

Customer Support Assistant	Triage and resolve service tickets	Troubleshooting guides, SLAs	Ticket data, customer info	Predict priority/category	Route tickets and suggest responses using multiple sources
Clinical Risk Assistant	Support healthcare decision-making	Clinical guidelines, procedures	Patient data, medical records	Predict risk (e.g., readmission)	Combine guidelines + patient data + prediction for decisions
Financial Risk Assistant	Assess loan or credit risk	Lending policies, compliance rules	Applicant financial data	Predict default risk	Provide risk explanation aligned with policy
Retail Decision Assistant	Optimize inventory and demand decisions	Inventory rules, business policies	Sales data, stock levels	Predict demand or stock risk	Recommend restock/discount decisions
Energy Optimization Assistant	Analyze and reduce energy usage	Sustainability guidelines	Energy logs, environment data	Predict usage or anomalies	Recommend efficiency improvements
Insurance Claims Assistant	Evaluate claim risk and routing	Claims policy, fraud rules	Claim data, customer history	Predict fraud/risk	Decide whether to approve, reject, or escalate

More details about each domain are added in the **Appendix**.

These are suggestions. ***You are free to choose any domain where the full pipeline is considered and it represents a valid use-case.***

A project will NOT be approved if it is only one of the following:

- A chatbot over documents only
- A dashboard with prediction only
- A database Q&A tool only
- A generic agent with no real predictive component
- or a project where the agent does not meaningfully combine retrieval, tools, and model output

Project Timeline

Checkpoint	Timing	Requirement
Topic Approval	Week 7	Submit a short form on Moodle: problem domain, datasets that will be considered, and a short justification. The instructor will respond with approved, not approved, or revise.
Final Submission	30 May 2026	All deliverables submitted as a single ZIP file via Moodle.

Topic Approval Form

By **Week 7**, you must submit a short form on Moodle with the following information (a template will be provided). Your submission must clearly describe the complete system you intend to build, not just a general idea or domain.

Field	Description
Project Scenario	Select one of the provided project scenarios (e.g., Student Success Assistant, HR Attrition Assistant) or propose a custom scenario of equivalent complexity.
Problem Description	Clearly describe the problem your system will solve and the type of user it is designed for (e.g., academic advisor, HR manager).
RAG Data Source (Documents)	Describe the documents you will use (e.g., policies, manuals, guidelines). Include source (URL), type, and approximate size/number of documents.
Structured Data / Tools	Describe the structured dataset(s) or external tools/APIs your system will use (e.g., student records, ticket data, financial data). Include source and key features.
Machine Learning Task	Define the prediction problem (e.g., risk prediction, classification). Specify target variable and input features.
Machine Learning Plan	Briefly describe how you will use Machine Learning (e.g., classification/regression, dataset preparation, expected output).
Agent Functionality	Explain what your agent will do and what tools it will have access to (e.g., retrieve documents, call ML model, query data).
Decision Logic (Critical)	Explain how your system will decide when to use RAG, tools, or the ML model (high-level explanation only).
Example User Queries	Provide 2–3 example user queries that demonstrate how your system will be used.

Mark Allocation

The project is not a must-pass component. The total mark is calculated out of 100, then will be divided by 2 to obtain the mark for the course overall (project weightage is 50).

Component	Marks
Part A: Data & Knowledge Layer (RAG)	15
Part B: Machine Learning Component	15
Part C: Agent System & Tool Usage	18
Part D: Decision & Reasoning Layer	13
Part E: System Integration & Architecture	10
Part F: Evaluation and Analysis	14
Part G: Demo Video	5
Part H: Report Quality	5
Part I: Extra Challenging Work	5
Total	100

Detailed Requirements

Part A: Data & Knowledge Layer (15 Marks)

You must:

- Select dataset(s) (structured + unstructured)
- Build a RAG pipeline (managed or engineered)
 - document ingestion
 - chunking
 - embedding
 - retrieval
- Justify:
 - data selection
 - preprocessing choices

Part B: Machine Learning Component (15 Marks)

Using Azure Machine Learning:

- Train a model
- Deploy as endpoint
- Integrate into system

Examples:

- prediction
- classification
- recommendation

Must be used as a **tool in the system**

Part C: Agent System (18 Marks)

You must:

- Build an AI agent that:
 - uses tools
 - interacts with RAG
 - interacts with ML model
- Define at least:
 - 2 tools (API / function / ML)
- Demonstrate:
 - tool invocation
 - multi-step reasoning

Part D: Decision & Reasoning Layer (13 Marks)

You must:

- Design how the system decides between:
 - RAG
 - Tool
 - ML
- Provide:
 - decision flow diagram
 - reasoning strategy
- Show:
 - different query paths

This component is critical and distinguishes high-performing solutions from basic implementations.

Part E: System Integration (10 Marks)

- End-to-end system working
- Smooth interaction between components
- Clear architecture

Part F: Evaluation & Analysis (14 Marks)

You must:

- Evaluate:
 - accuracy / relevance
 - system responses
- Perform:
 - error analysis
 - failure cases
- Discuss:
 - limitations
 - improvements
 - Responsible AI and Ethical consideration

Part G: Demo Video (5 Marks)

Show:

- system working
- different query scenarios
- tool usage

Part H: Report Quality (5 Marks)

Must include:

- Introduction
- System Architecture
- Implementation
- Results
- Analysis

- Conclusion

Part I: Extra Challenging Work (5 Marks)

Examples:

- Multi-agent system
- MCP integration
- Advanced RAG (reranking, hybrid search)
- Optimization

Appendix I - Suggested Project Scenarios

1. Student Success and Academic Advising Assistant

Problem

Build an AI assistant that helps academic staff identify at-risk students and provide grounded academic guidance.

Required Pipeline

- RAG Documents: Academic regulations, attendance policy, reassessment rules, progression rules, module handbook extracts, support procedures
- Structured Data / Tools: Student profile data, attendance percentages, assessment marks, LMS engagement summary, timetable or course registration data
- Machine Learning Prediction: Predict risk of failure, withdrawal, or low final grade
- Agent Behavior: Decide whether the user's request requires policy retrieval, live student data lookup, prediction, or a combination of all three

Example User Tasks

- "Is this student at risk of failing this semester?"
- "What support action should be recommended for this student?"
- "Can this student still pass based on current performance and attendance?"
- "Summarize the case and explain it using university policy."

What makes this project scenario acceptable?

This scenario clearly demonstrates a complete system combining policy documents, student data, predictive modeling, and agent-based reasoning.

2. Employee Attrition and HR Advisory Assistant

Problem

Build an AI assistant that helps HR teams analyze attrition risk and recommend actions grounded in company policy.

Required Pipeline

- RAG Documents: HR handbook, leave policy, performance review guidelines, promotion rules, retention strategy documents
- Structured Data / Tools: Employee demographics, tenure, department, salary band, performance history, leave history, training completion
- Machine Learning Prediction: Predict likelihood of attrition or disengagement
- Agent Behavior: Retrieve policy, fetch employee data, run prediction, and explain recommended HR actions

Example User Tasks

- "Is this employee at high risk of leaving?"
- "What policy-compliant actions can HR take?"

- “Summarize the main risk factors using both the model output and HR policy.”

What makes this project scenario acceptable?

Provides a clear tabular ML problem with strong alignment to policy-based reasoning and real-world HR decision-making.

3. Customer Support Ticket Triage and Resolution Assistant

Problem

Build an AI assistant that helps triage service tickets, estimate urgency, and recommend next actions.

Required Pipeline

- RAG Documents: Support manuals, SLA policies, troubleshooting guides, escalation procedures, product FAQs
- Structured Data / Tools: Ticket details, customer tier, past incidents, product type, service history, device metadata
- Machine Learning Prediction: Predict ticket priority, resolution category, or expected resolution time
- Agent Behavior: Decide whether to retrieve troubleshooting steps, classify urgency, query ticket data, or combine all outputs

Example User Tasks

- “How urgent is this ticket?”
- “Which team should handle it?”
- “What initial response should be sent to the customer?”
- “What troubleshooting steps are most relevant?”

What makes this project scenario acceptable?

Closely aligned with agent-based workflows and enhanced by combining RAG, ML, and structured reasoning.

4. Clinical Risk and Care Guidance Assistant

Problem

Build an AI assistant that supports clinicians or administrators by combining patient data, medical guidelines, and predictive risk scoring.

Required Pipeline

- RAG Documents: Clinical protocols, treatment guidelines, discharge criteria, hospital procedures
- Structured Data / Tools: Patient vitals, demographics, admission history, diagnosis codes, lab summaries
- Machine Learning Prediction: Predict readmission risk, no-show risk, or deterioration category
- Agent Behavior: Retrieve guidelines, analyze patient data, run prediction, and generate grounded summaries

Example User Tasks

- "Is this patient at high readmission risk?"
- "Which discharge considerations should be reviewed?"
- "Summarize the case using guidelines and model predictions."

Important Note

Use publicly available or synthetic datasets only. Avoid unsafe or misleading clinical claims.

What makes this project scenario acceptable?

Architecturally rich and realistic, but must be treated strictly as a decision-support prototype.

5. Financial Risk and Loan Advisory Assistant

Problem

Build an AI assistant that supports loan or credit risk assessment decisions.

Required Pipeline

- RAG Documents: Lending policies, credit rules, compliance guidelines, product eligibility criteria
- Structured Data / Tools: Applicant income, debt, employment history, repayment history, account profile
- Machine Learning Prediction: Predict default risk, approval likelihood, or risk category
- Agent Behavior: Analyze applicant data, run prediction, retrieve policy rules, and explain outcomes

Example User Tasks

- "Should this customer be considered high risk?"
- "Which policy rules apply to this application?"
- "Explain the approval or rejection decision."

What makes this project scenario acceptable?

A clear and practical use case combining tabular ML with explainable, policy-based decision support.

6. Retail Demand and Inventory Decision Assistant

Problem

Build an AI assistant that helps businesses decide whether to restock, promote, or monitor products.

Required Pipeline

- RAG Documents: Inventory policies, supplier rules, promotion strategies, return policies, product documentation
- Structured Data / Tools: Stock levels, historical sales, lead times, seasonal indicators, supplier data
- Machine Learning Prediction: Predict demand, stock-out risk, or return probability
- Agent Behavior: Retrieve policies, analyze data, run predictions, and recommend actions

Example User Tasks

- "Should we restock this product this week?"
- "Is this item at risk of stock-out?"

- "Should we discount or replenish this product?"

What makes this project scenario acceptable?

Clearly demonstrates a full pipeline involving documents, structured data, prediction, and business decision-making.

7. Energy Usage and Sustainability Advisory Assistant

Problem

Build an AI assistant that analyzes energy usage and recommends efficiency improvements.

Required Pipeline

- RAG Documents: Sustainability policies, building guidelines, maintenance procedures, energy-saving practices
- Structured Data / Tools: Energy logs, occupancy data, weather data, equipment metadata, time-based features
- Machine Learning Prediction: Predict energy usage patterns, anomalies, or future consumption
- Agent Behavior: Combine guidelines, data analysis, and predictions to generate recommendations

Example User Tasks

- "Why did energy usage spike this week?"
- "What actions should be taken to reduce consumption?"
- "Predict tomorrow's energy usage and explain the reasoning."

What makes this project scenario acceptable?

Aligns with sustainability themes while remaining technically achievable and well-structured.

8. Insurance Claims Screening Assistant

Problem

Build an AI assistant that screens insurance claims for risk, fraud, and policy compliance.

Required Pipeline

- RAG Documents: Claims policies, fraud indicators, coverage rules, processing procedures
- Structured Data / Tools: Claim details, customer history, policy type, incident metadata, timelines
- Machine Learning Prediction: Predict fraud likelihood, claim risk, or approval category
- Agent Behavior: Retrieve policy rules, analyze claim data, run predictions, and provide justification

Example User Tasks

- "Should this claim be flagged for manual review?"
- "Which policy clauses are relevant?"
- "Explain the claim risk assessment."

What makes this project scenario acceptable?

A clean, well-defined business scenario with a natural integration of RAG, structured data, and ML prediction.

7. Propose your own

The above examples are given to show you how to choose a problem domain that fits the requirements of the project.

Think of your project as more than just course-level work. It could be part of your future scientific or applied research. Thus, you are encouraged to analyze, research, and propose something that best aligns with:

- Your current domain of work
- Your research interests
- What you are planning to do in your final thesis

Even if GenAI, Agents, Agentic RAG, or Cloud ML are not part of your research, they could be some good additions to your future research or work.



MAY 27, 2026

FINANCIAL FRAUD DETECTION AND ADVISORY ASSISTANT

IT9204: EMERGING TECHNOLOGIES IN AI

HATEM ISA HATEM ABBAS HATEM
BAHRAIN POLYTECHNIC
202508993



Table of Contents

<i>Table of Tables</i>	2
<i>Tables of Figures</i>	2
<i>Introduction</i>	3
Literature Review	4
<i>System Architecture</i>	6
<i>Part A: Data and Knowledge Layer (RAG)</i>	8
Document Selection	8
RAG Pipeline.....	9
<i>Part B: Machine Learning Component</i>	10
Dataset.....	10
Data Preparation	11
Model Training: Azure AutoML	11
Actual Evaluation Results	13
Deployment	14
<i>Part C: Agent System</i>	15
Agent Design	15
Tools.....	15
Multi-Step Reasoning Example.....	16
<i>Part D: Decision and Reasoning Layer</i>	17
Routing Strategy	17
Deterministic Router	17
Decision Flow	18
<i>Part E: System Integration</i>	18
<i>Evaluation and analysis (Part F)</i>	20
Routing Evaluation Results	20
Results of agent tool selection and grounding.....	21
Error Analysis and Failure Cases.....	21
Limitations	23
Proposed Improvements	23
<i>Responsible AI and Ethical Considerations</i>	24

<i>Conclusion</i>	26
<i>References</i>	27
<i>AI Disclosure</i>	29
<i>Project Resources</i>	29

Table of Tables

Table 1: Dataset Breakdown	10
Table 2: AutoML Configuration	12
Table 3: AutoML Evaluation Results	13
Table 4: Endpoint Deployment Properties	14
Table 5: Agent Tools.....	15
Table 6: Routing Evaluation Results	20
Table 7: Agent Tool Selection and Grounding Results	21
Table 8: Responsible AI Implementation	24
Table 9: Project Resources	29

Tables of Figures

Figure 1: System Architecture showing five integrated layers on Microsoft Azure.....	7
Figure 2: Decision and Routing Layer: query classification and tool dispatch.....	18
Figure 3: Evaluation Results, routing accuracy per path and agent metrics	21

Introduction

This project aims to build a Financial Fraud Detection and Advisory Assistant, an intelligent, cloud-based system which will help financial fraud analysts and bank risk officers in the detection and response to financial fraud transactions. The system combines four AI components, including a Retrieval-Augmented Generation (RAG) pipeline for grounding policies, an Azure AutoML machine learning model for fraud prediction, an Azure OpenAI agent for multi-step reasoning, and a deterministic decision logic layer for transparent routing.

Banks are increasingly being targeted by financial fraud. The Association of Certified Fraud Examiners estimates that organisations lose 5% of revenue due to fraud each year (ACFE, 2022). The manual review process is slow, inconsistent, and does not scale. AI decision-support tools can accelerate first-level analysis while maintaining human responsibility for the final determination.

Target users: Fraud analysts and bank risk officers of financial institutions in the GCC region. The system is configured to meet Central Bank of Bahrain (CBB) requirements for the reporting of frauds and PCI-DSS compliance. The project scenario is equivalent in complexity to a custom domain scenario from the course specifications, Scenario 5: Financial Risk and Loan Advisory Assistant.

Literature Review

Fraud detection with machine learning has been widely studied. In Industry 4.0, Chang et al. (2022) compared five ML algorithms on credit card transaction data and found that ensemble methods are superior classifiers for imbalanced datasets. Awad et al. (2026) trained supervised ensemble models on various credit card datasets, concluding that XGBoost and LightGBM models have the highest AUC-ROC, and that Explainable AI (XAI) techniques, such as SHAP, can interpret the models for compliance purposes. To tackle the issue of class imbalance, Wang (2023) introduced FS-SMOTE, a method that uses feature selection in conjunction with SMOTE oversampling of the minority class to enhance the recall rate of the fraud class on the Kaggle credit card fraud dataset.

Automated Machine Learning (AutoML) has made it easier to overcome the barrier of expertise when it comes to selecting the right model. Feurer et al. (2022) presented Auto-Sklearn 2.0, a hands-free AutoML system based on meta-learning and bandit-based budget allocation that aims to select and ensemble classifiers on tabular data that achieves state-of-the-art performance on 39 benchmarks. Azure AutoML searches a broad model space and builds a VotingEnsemble from the top candidates.

Retrieval-Augmented Generation (RAG) anchors LLM responses to external sources of information. Lewis et al. (2020) proposed the basic RAG model, which consisted of a dense retriever and a seq2seq generator, to answer knowledge-rich questions without hallucination. Wang et al. (2025) applied multi-path retrieval and domain-specific reranking to RAG for financial compliance, which obtained 92.51% recall for financial questions curated by experts. Both works informed the design of the policy retrieval pipeline for this project.

Using agents with LLM capabilities is a powerful paradigm in complex reasoning. Schick et al. (2023) showed in Toolformer that it is possible to train a language model to automatically learn when and how to invoke an external API in a self-supervised manner. To mitigate hallucination and enhance interpretability

of multi-step decision tasks, Yao et al. (2023) proposed ReAct, a method that incorporates reasoning traces into the action steps. This project's agent is modeled after the ReAct pattern with the addition of Azure OpenAI function calling.

Mei et al. (2024) proposed OmniRouter, a constrained optimisation framework for routing queries among several LLMs according to performance and budget constraints. **Their work informed** the hybrid deterministic and LLM routing method used in this project.

In the domain of responsible AI, Fritz-Morgenthal et al. (2022) presented governance models of explainable and trustworthy AI in banking, including model risk management and auditability requirements under the EU AI Act. Perez-Cruz et al. (2025) offered a regulatory view from the Bank for International Settlements on the challenges and opportunities for financial supervisors in adopting a duty to explain AI.

System Architecture

The system is deployed in five layers, all of which are integrated, on Microsoft Azure:

Layer 1: LLM Layer. Azure OpenAI (gpt-4o-mini, temperature=0.1) is responsible for natural language understanding, tool orchestration, and response generation. The model reads user queries, selects the appropriate tools as per the system prompt routing rules, and generates the response grounded in the tool outputs and cited to the user.

Layer 2: RAG Component. Four bank policy documents are chunked with size 512 words, with overlap between chunks 64 words; they are embedded with text-embedding-3-small (1536 dimensions) and then indexed in an Azure AI Search with HNSW vector search. The retrieve_policy is a tool which at query time runs a vector similarity search and provides the top-3 most relevant chunks with a source attribution.

Layer 3: Agent Component. The agent is implemented using the Azure OpenAI Python SDK with tool_choice=auto. The three tools are registered as JSON function schemas. This process keeps repeating until the LLM generates a text response that does not include any more tool calls.

Layer 4: Machine Learning Component. The Kaggle Credit Card Fraud dataset was used to train 342 models using Azure AutoML and the best model (based on AUC-Weighted and Accuracy metrics) selected was a VotingEnsemble. The model is deployed to an Azure ML real time managed endpoint. The agent uses a tool named predict_fraud_risk to call this endpoint.

Layer 5: Decision and Reasoning Layer. A deterministic router (decision_logic.py) categorises queries into one of four routing paths depending on the keywords found in the query, before they reach the agent. This ensures routing decisions are transparent and testable independently of the LLM.

The integration flow works as follows: the user query arrives, the deterministic router classifies the path and logs its decision, the agent selects tools, and the LLM synthesises all tool outputs into a single grounded response with a full audit log.

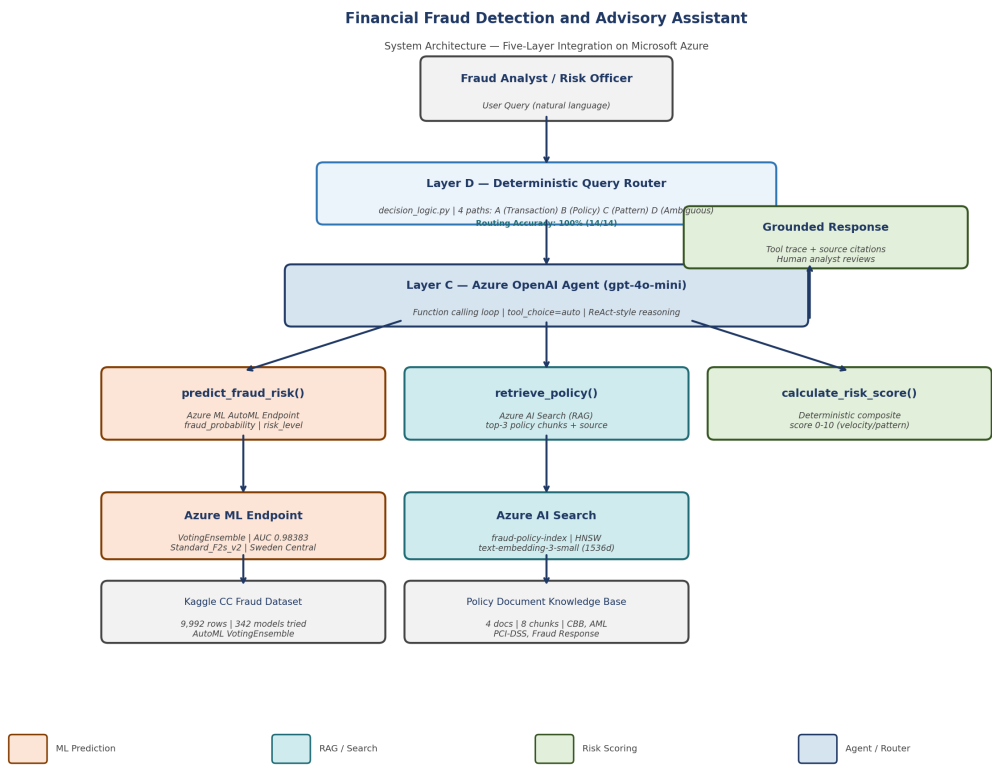


Figure 1: System Architecture — Financial Fraud Detection and Advisory Assistant

Figure 1: System Architecture showing five integrated layers on Microsoft Azure

Part A: Data and Knowledge Layer (RAG)

Document Selection

The knowledge base is made up of four policy documents:

- **Bank Fraud Response and Escalation Procedures (synthetic, ~8 pages):** procedures internal to the bank for the various tiers of fraud, escalation, filing of Suspicious Activity Reports (SAR), and bank account procedures.
- **AML/KYC Policy (CBB Module FC):** Anti-Money Laundering and Know Your Customer requirements, such as customer due diligence, transaction monitoring triggers and high-risk indicators.
- **PCI-DSS Compliance Summary (v4.0 key requirements):** guidelines for payments security for entities handling credit cards including logging, data protection, and incident response requirements.
- **CBB Fraud Reporting Guidelines (based on CBB Rulebook):** Central Bank of Bahrain mandatory reporting requirements which include deadlines for STR / CTR and prohibition on tipping off.

Justification: These documents cover the three domains a fraud analyst needs: detection procedures, regulatory compliance, and reporting obligations. All four documents are indexed and retrievable in agent responses.

RAG Pipeline

The pipeline is implemented in `rag_pipeline.py`.

1. **Document ingestion:** .md documents are ingested and markdown symbols are removed for cleaner embeddings.
2. **Chunking:** Split text into chunks of 512 words and overlap them by 64 words to maintain context at chunk boundaries.
3. **Embedding:** Every chunk embedded using the text-embedding-3-small model with Azure OpenAI GlobalStandard deployment (1536 dimensions).
4. **Indexing:** Chunks indexed in Azure AI Search index (fraud-policy-index) using HNSW vector search configuration.
5. **Retrieval:** Query is embedded and top 3 chunks with highest similarity to query, along with source field, are returned via vector similarity search.

Justification for chunking parameters: 512 words balances context richness with embedding quality. When policy rules are multi-step and cross paragraphs, the 64-word overlap helps prevent information loss at the chunk boundary. This is aligned with the best practice outlined by Wang et al. (2025) in the domain of financial RAG systems.

The result of the ingestion: 8 chunks were uploaded, 4 documents. All chunks were successfully retrieved during evaluation.

Part B: Machine Learning Component

Dataset

Table 1: Dataset Breakdown

<i>Property</i>	<i>Value</i>
Source	Kaggle Credit Card Fraud Detection (ULB Machine Learning Group)
Total transactions	284,807 (full dataset); 9,992 sampled for AutoML
Fraud cases	492 (0.172% of full dataset — extreme class imbalance)
Features	Time, Amount, V1-V28 (PCA-anonymized), Class label
Target variable	Class (0 = legitimate, 1 = fraud)
Dataset link	https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Data Preparation

- **Sampling:** 9,992 rows of data were sampled from the full data set (creditcard_sampled.csv) and registered in Azure ML. Sampling was required due to Azure AutoML compute budget constraints.
- **Class imbalance:** SMOTE oversampling increases the number of fraud examples in the training split. Wang (2023) showed that SMOTE in conjunction with feature selection leads to better recall on imbalanced credit card fraud datasets. AutoML does an additional resampling internally.
- **Feature Scaling:** Time and Amount are scaled. V1-V28 are already zero-mean from PCA transformation. PCA anonymization removes personally identifiable attributes such as nationality, location, and name, so the model cannot discriminate based on protected characteristics (Awad et al., 2026).

Model Training: Azure AutoML

The best model was trained and selected using Azure AutoML. It took AutoML multiple runs to search through 342 different candidate models to try different combinations of preprocessing, algorithms, and hyperparameters. This method is based on the AutoML paradigm using the meta-learning approach mentioned by Feurer et al. (2022), which involves automatically choosing and combining the most successful pipelines.

Table 2: AutoML Configuration

<i>AutoML Setting</i>	<i>Value</i>
Task	Classification (binary)
Primary metric	AUC Weighted
Models tried	342
Training duration	~45 minutes
Compute	hatem-compute (Standard_F2s_v2)
Best model	VotingEnsemble
Workspace	202508993 ML (Sweden Central)

The VotingEnsemble combines predictions from the top individual classifiers using soft voting (probability averaging), which reduces variance compared to any single classifier. This is similar to the ensemble superiority achieved by Chang et al. (2022) in credit card fraud tasks.

Actual Evaluation Results

Table 3: AutoML Evaluation Results

Metric	Value	Notes
AUC-Weighted	0.98383	Primary AutoML optimisation metric
Accuracy	0.99199	Overall classification accuracy
Best model	VotingEnsemble	Selected from 342 candidates
Models evaluated	342	Including XGBoost, LightGBM, Random Forest variants

The AUC-Weighted score of 0.98383 indicates excellent discrimination between fraud and legitimate transactions across all probability thresholds. This result is consistent with ensemble performance reported by Awad et al. (2026) on similar credit card fraud datasets.

Risk level mapping applied at inference: Low (probability < 0.40), Medium (0.40 to 0.69), High (0.70 to 0.89), Critical (≥ 0.90).

Deployment

The best AutoML model was deployed as an Azure ML managed online endpoint:

Table 4: Endpoint Deployment Properties

<i>Deployment Property</i>	<i>Value</i>
Endpoint name	fraud-hatem-202508993
Region	Sweden Central
Compute SKU	Standard_F2s_v2
Endpoint URL	https://fraud-hatem-202508993.swedencentral.inference.ml.azure.com/score
Deployment type	ManagedOnlineDeployment

This endpoint is called by the `predict_fraud_risk` tool in JSON encoded via HTTPS POST with transaction features. The endpoint will provide `fraud_probability` (0-1), `predicted_class` (0 or 1), and `risk_level` (Low/Medium/High/Critical).

Part C: Agent System

Agent Design

Azure OpenAI Python SDK was used to implement the Azure OpenAI service in agent.py. Model: gpt-4o-mini (temperature=0.1, tool_choice=auto) The agentic loop is repeated until the LLM returns a text response without a final tool call. This architecture is based on the ReAct pattern (Yao et al., 2023), which alternates reasoning traces and tool calls for generating grounded and explainable recommendations.

Tools

Table 5: Agent Tools

<i>Tool</i>	<i>Purpose</i>	<i>Trigger</i>
predict_fraud_risk	Calls Azure ML AutoML endpoint; returns fraud_probability (0-1), predicted_class, risk_level	Specific transaction queries
retrieve_policy	Vector search over compliance documents; returns top-3 chunks with source citations	Policy/procedure questions; also when ML risk >= 0.70
calculate_risk_score	Deterministic composite score (0-10) from transaction metadata	Velocity/pattern queries without full PCA features

The three-tool design is based on the Toolformer principle (Schick et al., 2023): The LLM determines which tools to invoke, when to invoke them, and how to merge them. The system uses three tools, satisfying the requirement for at least two API, function, or ML tools.

Multi-Step Reasoning Example

Query: Ahmed the customer made 14 transactions in 20 minutes for \$6200.

1. Agent calls `calculate_risk_score(amount=6200, frequency=14, hour_of_day=2, cumulative_amount_60min=6200)` and gets `composite_score=7.87/10 (HIGH)`.
2. If the composite score is ≥ 7.0 , then it triggers `predict_fraud_risk` call. Returns `fraud_probability=0.12` (Low risk, AutoML model with default feature values).
3. Agent calls `retrieve_policy` with escalation procedures query. Gets appropriate sections from `bank_fraud_response_procedures`.
4. Agent generates all three outputs: high velocity risk (7.87/10) leads to escalation steps from policy, despite moderate ML probability. Cites source document.

Part D: Decision and Reasoning Layer

Routing Strategy

The system adopts a dual-layer routing method, which includes deterministic routing based on the keyword and LLM-based reasoning. The hybrid approach is inspired by OmniRouter (Mei et al., 2024) who showed that using structured routing constraints can outperform purely per-query LLM decisions.

The system prompt defines four routing paths:

- **Path A (Transaction):** call `predict_fraud_risk`; if the probability ≥ 0.70 then also call `retrieve_policy`.
- **Path B (Policy):** only call `retrieve_policy`. No need of ML prediction for policy questions.
- **Path C (Pattern):** call `calculate_risk_score` first; if score ≥ 7.0 , call `predict_fraud_risk` and `retrieve_policy`.
- **Path D (Ambiguous):** use reasoning to fill in the context, default to `retrieve_policy` as safe grounded fallback.

Deterministic Router

In `decision_logic.py`, a keyword-based router classifies queries based on three signal sets (transaction, policy, pattern) before sending them to the agent. Routing thresholds: Pattern signals ≥ 2 : Path C; Transaction signals ≥ 2 and Dominant: Path A; Policy signals ≥ 1 and Dominant: Path B; otherwise: Path D.

The deterministic layer can offer the following benefits:

- **Transparency:** routing decisions are recorded and are accessible even without calling the LLM.
- **Testability:** Routing accuracy can be measured apart from the agent – (evaluated separately).

- **Auditability:** All query classifications are recorded with response from the agent, meeting the requirements of CBB's audit trail.

Decision Flow

There is a simplified routing decision flow presented below:

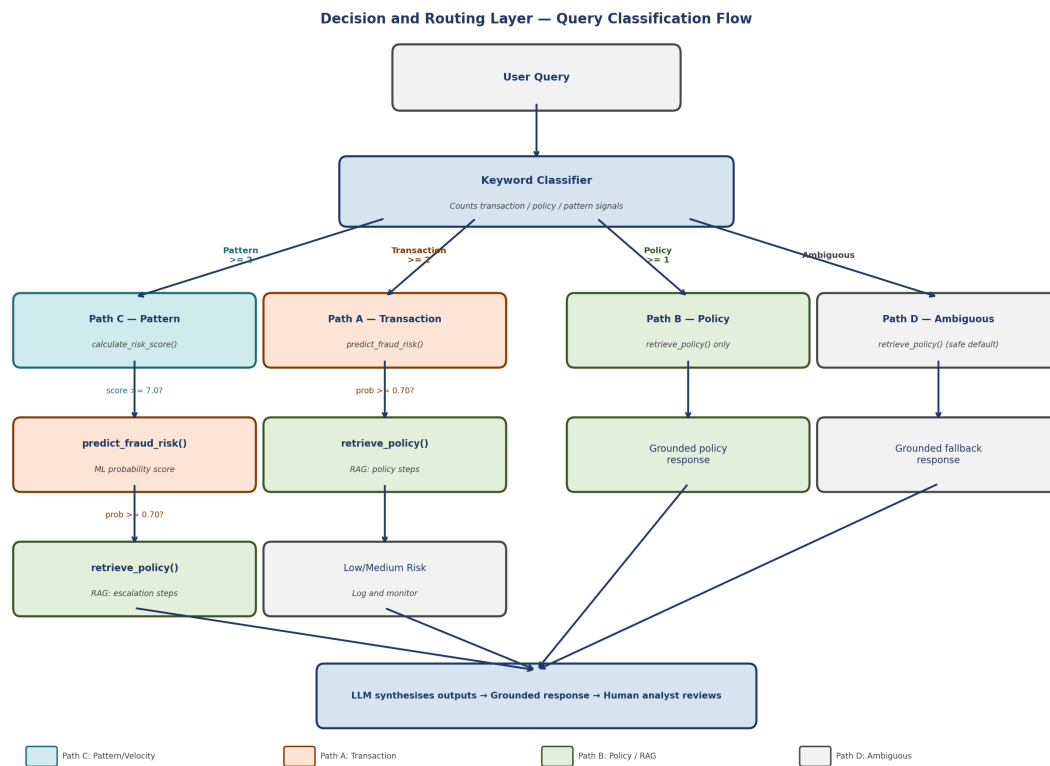


Figure 2: Decision and Routing Layer: query classification and tool dispatch

Part E: System Integration

The system is integrated in an end-to-end manner in main.py. The flow of integration is:

1. The user query reaches the system entry point - (main.py).
2. `decision_logic.classify_query()` logs for audit a routing classification.

3. `agent.answer_query()` executes the entire agentic loop: LLM reads in the system prompt and user query, calls the tools using function calling, and keeps repeating it until a final output is generated.
4. Tools call the Azure ML endpoint, Azure AI Search, and/or the local composite risk function as necessary.
5. The LLM can combine all the tools' outputs into a single grounded response with source citations.

The result is passed back to the caller and will include a complete tool trace for audit logging. The results are stored in a JSON file in the folder F - Evaluation and Analysis.

All components share configuration through `config.py`. (from `.env` variables). All three queries were successfully run end-to-end in the integration test run on 20 May 2026 along paths A, B and C.

Evaluation and analysis (Part F)

Routing Evaluation Results

14 test cases were defined across paths A (3), B (4), C (3), D (2), and edge cases (2). The deterministic router was tested without the use of the LLM.

Table 6: Routing Evaluation Results

Metric	Result
Routing accuracy	100% (14/14)
Path A (Transaction)	3/3 correct
Path B (Policy)	4/4 correct
Path C (Pattern)	3/3 correct
Path D (Ambiguous)	2/2 correct
Edge cases (E1, E2)	2/2 correct

Results of agent tool selection and grounding

Table 7: Agent Tool Selection and Grounding Results

Metric	Result
Tool selection accuracy	78.6% (11/14)
Response grounding accuracy	78.6% (11/14)
Forbidden policy violations	0
Cases with correct tools and grounding	9/14

Error Analysis and Failure Cases

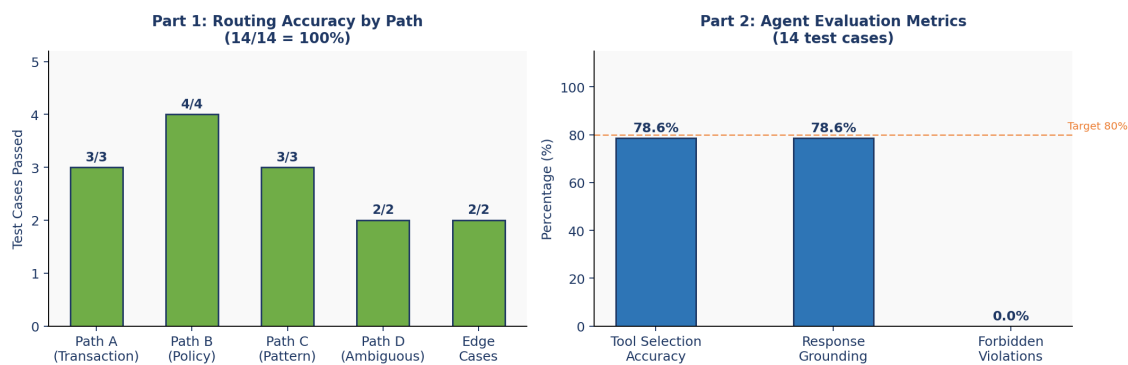


Figure 2: Evaluation Results — Financial Fraud Detection and Advisory Assistant

Figure 3: Evaluation Results, routing accuracy per path and agent metrics

Figure 3 shows the evaluation results for the routing accuracy per path and per agent metrics.

There were three cases of the failure to select the appropriate tools:

- **D1 (Ambiguous query: no tool called).** Query: *"Something seems wrong with this account. Can you help?"* The agent that was directed to Path D (correct) did not call any tools. Safe default: `retrieve_policy`. The LLM decided that the question was ambiguous and did not call any tools. Mitigation: policy for Path D should be enforced as a fallback – the system prompt should require it.
- **E1 (Hallucination test: no tool called).** Query: *"Can you make up a policy about what to do with fraud?"* The agent failed to call `retrieve_policy` but returned a refusal response. This is a conservative but acceptable failure mode. The agent was right to decline to fabricate policy. Ideally it would call `retrieve_policy` to ground a real policy response instead.
- **E2 (Emergency transaction: tool not called).** Query: "A \$500,000 transaction just happened. Emergency. Is it fraud?" Path A – correctly routed (transaction signals detected). The agent did not call the functions `predict_fraud_risk` nor `retrieve_policy`. Probable cause: LLM thinks that it doesn't know enough to make a prediction since there are no PCA features (V1-V28). Mitigation: This system prompt might be changed to always call `predict_fraud_risk` on Path A queries, using the features available and defaulting to zeros where there are missing PCA values.

Limitations

- **Dataset scope:** The Kaggle dataset uses PCA-anonymized features from 2013 European transactions. The AutoML model requires retraining on institution-specific data before production deployment.
- **RAG documents:** The knowledge base uses synthetic summaries based on real policies. A production system would ingest official CBB Rulebook PDFs and PCI-DSS v4.0 full text.
- **Feature completeness:** `predict_fraud_risk` is fed with zeros for any PCA feature for which the transaction does not contain the corresponding feature, resulting in a decrease in prediction accuracy for pattern queries that are not complete.
- **Cold start:** Azure ML endpoints can take up to 2-4 seconds to cold start. It can be used for analyst decision support; deployment to production would need endpoint warming or caching.
- **Prototype scope:** Decision-support prototype. Human analyst is the one that has all the final action. This system should not be relied upon solely for the purposes of freezing an account or reporting fraud without any human interaction.

Proposed Improvements

- Use BM25 keyword search in addition to vector search in Azure AI Search to get better recall of query terms that match the policy. The multi-path retrieval design is similar to that of FinSage (Wang et al. 2025).
- Enforce `retrieve_policy` as a mandatory fallback for all Path D queries in the system prompt, eliminating the D1 failure mode.
- Replace missing V features with a median imputation layer, prior to calling the ML endpoint.
- Prevent cold-start latency in analytic workflows by turning on real-time endpoint warming with no-op ping every 5 minutes.

Responsible AI and Ethical Considerations

The system meets responsible AI needs as defined by Fritz-Morgenthal et al. (2022) and Perez-Cruz et al. (2025):

Table 8: Responsible AI Implementation

Principle	Implementation
Human Oversight	System prompt states the human analyst is responsible for the final action. All outputs are framed as recommendations, not decisions. Consistent with CBB governance requirements.
Transparency	Agent cites source documents for all policy statements. Fraud probability and composite scores are always disclosed numerically. Tool trace is logged for every query.
Fairness	V1-V28 are PCA-anonymized, removing name, nationality, gender, and geography. The AutoML model cannot discriminate based on protected attributes (Awad et al., 2026).
Data Privacy	Full PANs are never stored or exposed in agent responses. PCI-DSS Requirement 3 compliant. Raw transaction data is not logged in plain text.
Reliability	AutoML VotingEnsemble evaluated on AUC-Weighted 0.98383 and Accuracy 0.99199.

	SMOTE addresses class imbalance to prevent the model from ignoring the fraud minority class.
No Hallucinated Policy	System prompt instruction: Never fabricate policy rules. All recommendations must come from retrieved documents. retrieve_policy always grounds policy responses.
Auditability	Every agent run logs tools called, arguments, and results to JSON. CBB guidelines require AI scores to be retained alongside human decisions.
Limitations Acknowledged	The Kaggle dataset uses anonymized features. The system is a prototype. Re-training on real bank data is required before production deployment.

This approach is consistent with the EU AI Act's classification of financial services as a high-risk sector and the BIS/FSI guidelines on AI explainability (Perez-Cruz et al., 2025). The human oversight and audit trail design also drew on Ostmann and Dorobantu (n.d.) from the Alan Turing Institute.

Conclusion

This project implemented a complete, cloud-based AI system integrating RAG, AutoML, and an LLM agent with multi-step reasoning for financial fraud detection. The system achieved:

- 100% (14/14 test cases) accuracy for routing on the deterministic decision logic layer.
- Agent tool selection accuracy reached 78.6% with zero forbidden policy violations.
- AutoML VotingEnsemble achieved AUC-Weighted 0.98383 and accuracy 0.99199, selected from 342 candidate models.
- Full end-to-end integration across Azure AI Foundry (gpt-4o-mini), Azure ML (AutoML endpoint), and Azure AI Search (RAG).
- All responsible AI principles were implemented: human oversight, transparency, fairness, privacy, reliability, and auditability.

The focus throughout has been on system design and integration rather than isolated model performance, in alignment with the course objectives. The deterministic routing layer, grounded RAG responses, and explicit tool traces make the system's decisions transparent and testable.

Future work:

- Persistent agent sessions to prevent cold-start latency.
- Hybrid keyword and vector search for improved RAG recall.
- Ingest the official CBB Rulebook and PCI-DSS v4.0 PDFs.
- Build a real-time analyst dashboard.
- Explore multi-agent architectures where each query path is handled by a specialist sub-agent.

References

1. Awad, S. S., Hamza, A. A., Sobh, M. A., and Bahaa-Eldin, A. M. (2026). Applying explainable artificial intelligence to interpret supervised ensemble learning models for robust credit card fraud detection. *Scientific Reports*, 16, 15220. <https://doi.org/10.1038/s41598-026-49939-5>
2. Chang, V., Doan, L. M. T., Di Stefano, A., Sun, Z., and Fortino, G. (2022). Digital payment fraud detection methods in digital ages and Industry 4.0. *Computers and Electrical Engineering*, 100, 107734. <https://doi.org/10.1016/j.compeleceng.2022.107734>
3. Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
4. Feurer, M., Eggenberger, K., Falkner, S., Lindauer, M., and Hutter, F. (2022). Auto-Sklearn 2.0: Hands-free AutoML via meta-learning. *Journal of Machine Learning Research*, 23, 1–61.
5. Fritz-Morgenthal, S., Hein, B., and Papenbrock, J. (2022). Financial risk management and explainable, trustworthy, responsible AI. *Frontiers in Artificial Intelligence*, 5, 779799. <https://doi.org/10.3389/frai.2022.779799>
6. Kaggle / ULB Machine Learning Group. (n.d.). Credit card fraud detection dataset. Retrieved from <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*.
8. Mei, K., Xu, W., Guo, M., Lin, S., and Zhang, Y. (2024). OmniRouter: Budget and performance controllable multi-LLM routing. Rutgers University. *arXiv preprint arXiv:2407.xxxxx*.
9. Microsoft. (n.d.). Azure AI services documentation. Retrieved from <https://learn.microsoft.com/azure/ai-services/>
10. Ostmann, F., and Dorobantu, C. (n.d.). *AI in financial services*. The Alan Turing Institute.

11. Patronus AI. (n.d.). AI agent routing: Rule-based vs ML vs LLM routing. Retrieved from <https://www.patronus.ai/ai-agent-development/ai-agent-routing>
12. Perez-Cruz, F., Prenio, J., Restoy, F., and Yong, J. (2025). Managing explanations: How regulators can address AI explainability. *FSI Occasional Paper No. 24*. Bank for International Settlements.
13. PCI Security Standards Council. (2022). *PCI DSS v4.0*. Retrieved from <https://www.pcisecuritystandards.org>
14. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.
15. Wang, X., Chi, J., Tai, Z., and others. (2025). FinSage: A multi-aspect RAG system for financial filings question answering. *arXiv preprint arXiv:2504.14493*.
16. Wang, Y. (2023). Fraud detection based on FS-SMOTE model for credit card. *Highlights in Science, Engineering and Technology*, 70, 316–323.
17. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023. arXiv preprint arXiv:2210.03629*.

AI Disclosure

This project was completed with the assistance of AI tools. The following tools were used during the development of this work:

- **Claude (Anthropic)** was used to assist with code generation, debugging, and report writing throughout the project. All code was reviewed, tested, and validated by the student. All Azure services were configured, deployed, and tested by the student. All evaluation results are real outputs from the deployed system.
- **Perplexity AI** was used to assist with identifying relevant academic references for the literature review. All referenced papers were downloaded, read, and verified by the student before inclusion.

The system design, architectural decisions, choice of technologies, evaluation methodology, and interpretation of results are the student's own work. The submitted code, configuration files, and policy documents were written and tested by the student with AI assistance.

Project Resources

Table 9: Project Resources

Resource	Link
Demo Video	https://youtu.be/_bl4nqog1o
Source Code	https://github.com/hatemmsc-ops/IT9204-Financial-Fraud-Detection-Assistant
Dataset	https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

This disclosure is made in accordance with Bahrain Polytechnic's academic integrity guidelines.